

Contenuti

Sprint 0:

1. Partiamo dall' **insieme di requisiti** forniti dal committente espressi in linguaggio naturale.
2. Svolgiamo un' **analisi di questi requisiti** , con l'obiettivo di esprimerli in linguaggio comprensibile a una macchina, e quindi di ottenere:
 - un **modello** dell'architettura del sistema desunta dai requisiti
 - un (primo) **TestPlan** di **Functional Tests** (capitolo 5.10), con particolare attenzione alle **User Actions** , ovvero quelle azioni (esprese nei requisiti) con cui l'utente si aspetta di poter interagire con il sistema.
3. Impostiamo un **piano di lavoro** che mira a definire:
 - una prima **architettura logica** del sistema come risultato dall'analisi
 - la **definizione del primo SPRINT** della produzione

(da [Il nostro metodo di lavoro](#), capitolo 5.10.1)

Sarà inoltre obiettivo dello **Sprint0**:

- Evidenziare quali componenti sono forniti/indicati dal committente e quali invece bisogna sviluppare, dando indicazioni sulla natura software dei componenti stessi.
- Evidenziare il **core business** del sistema

(da [Lo Sprint0](#), capitolo 32.1.1)

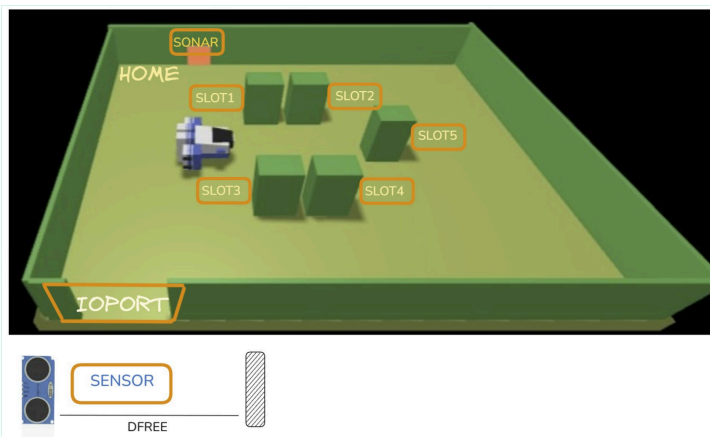
Inoltre, visto che la nostra software house si occupa già di robotica, sarà importante esplicitare l'esistenza di **software precedentemente sviluppati** che, da una prima lettura dei requisiti, ci sembra potrebbero essere utili nelle fasi successive.

Requisiti

A Maritime Cargo shipping company (from now on, simply **company**) intends to automate the operations of load of **containers** in the ship's cargo hold (or simply **hold**). To this end, the company plans to employ a *Differential Drive Robot* (from now, called **cargorobot**). The **hold** is a rectangular, flat area with an Input/Output port (**IOPort**). The area provides 4 *slots* to store the containers and a slot named **slot5**.

In the picture below:

- The **slots1-4** depict the hold areas reserved to store one container each
- The **slot5** depicts an area, where the cargorobot must temporarily store a container, before to place it in one of the slots1-4. During temporary storage, a 'marker' device labels the container with an identification barcode and signals when this marking activity is completed
- The **IOPort** is a device with a **pushbutton** and a **display**. The pushbutton is pressed by the customer in order to send a request to load a container on the cargo. The display is used to show the answer to the request and to show the current state of the hold
- The **sensor** associated to the IOPort is a device (a sonar) used to detect the presence of a container, when it measures a distance D , such that $D \leq D_{FREE}/2$, during a reasonable time (e.g. 3 secs)



The company asks us to build service named **cargoservice** that should work as follows.

The *cargoservice* is able to receive a **request to load** a container sent by some customer by using the *pushbutton* of the *IOPort*.

- It sends the answer **retrylater**, if the **IOPort** is currently occupied by a container or if the system is *Out of service*
- It rejects the request when the hold is already full, i.e. the **slots1-4** are already occupied.
- Otherwise, it considers the system as **engaged**, detects a free slot and returns as answer the name of such a reserved slot. While engaged, the system must blink a Led.

When the *request to load* is accepted, the customer must move the container in the *sensor* area within prefixed amount of time (e.g. 30 secs), otherwise the systems becomes **disengaged**. Then, the *cargoservice* uses the *cargorobot* to move the container from the *IOPort* to the *slot5* (for marking the container) and then to the reserved slot.

The service must also show on the *display* on the *IOPort*:

- the current state of the *hold*
- the message '**Service working**', when it is all is going well
- the message '**Out of service**' if the *sonar sensor* measures a distance $D \geq DFREE$ for at least 3 secs (perhaps a failure of the sonar).

Premessa: utilizzo di QAK

L'analisi dei requisiti si fonderà principalmente sul concetto di attore e sull'utilizzo del [metamodello QAK](#).

Questo perchè, già da requisiti, risulta che il sistema sia caratterizzato da diversi componenti che necessitano di comunicare fra loro.

Ad esempio, parlando col committente, ci ha specificato come il **sensor** associato alla **IOPort** sia stato pensato come da montare fisicamente su un dispositivo Raspberry, mentre il servizio **cargoservice** descritto nei requisiti sia stato pensato per essere eseguito su un PC.

Per questo riteniamo necessario spostarci da una rappresentazione che fa riferimento alla programmazione ad oggetti, a una che sfrutta un **DSL**, quindi un linguaggio che ci permette di ridurre l'abstraction gap, introducendo entità che possono di rappresentare :

- I diversi **contesti** in cui il cliente si aspetta che i vari componenti vengano eseguiti.
- I componenti del sistema come **attori**, quindi entità caratterizzate non solo dai dati ma anche dalla comunicazione con le altre entità del sistema.
- I **messaggi** scambiati dagli attori del sistema.

Tutto questo senza la necessità di utilizzare librerie o implementazioni specifiche, semplificando così la lettura e la comprensione futura.

Pensiamo inoltre che l'utilizzo di **QAK** ci permetta di pensare a ogni componente del sistema come un **microservizio**, ottenendo così piena technology independence, ovvero la possibilità di non essere costretti a una singola tecnologia per lo sviluppo dell'intero sistema, ma invece di essere liberi di scegliere la tecnologia più appropriata per ogni componente senza paura di complicare la comunicazione e lo sviluppo.

Domande al committente

1. Quanto dura il periodo di marcatura del container presso lo Slot 5?
Risposta del committente: è possibile assumere un valore fisso di 2 secondi.
2. Su quanti e quali nodi di calcolo verrà eseguito il sistema?
Risposta del committente: Un computer che gestisce il cargoservice, uno che gestisce l'IOPort, un Raspberry che gestisce il sensore e il led, e il computer di bordo del cargorobot.
3. Come è realizzata la IOPort?
Risposta del committente: la IOPort è implementata attraverso una GUI Web; il pushbutton e display sono degli elementi interattivi sulla pagina web.
4. Come rappresentare visivamente lo stato del sistema sul display?
Risposta del committente: Basta rappresentare l'occupazione degli slot sulla GUI Web.
5. Come avviene il ripristino del sistema dallo stato Out-of-Service?
Risposta del committente: Il sistema deve ritornare operativo non appena il sensor rileva $D \leq D_{FREE}$
6. E' previsto lo svuotamento degli slot?
Risposta del committente: No, uno slot pieno va considerato permanentemente pieno.

Analisi dei Requisiti

Cargorobot

Cargorobot da requisiti è un DDR, quindi il robot che si occupa di spostare i container dalla porta di carico allo slot 5 per la marcatura, e successivamente allo slot riservato. In questa fase di modellazione, senza pensare a un'implementazione vera e propria di questo robot, pensiamo di modellarlo come QActor **cargorobot** all'interno di un contesto `ctx_robot` separato da tutti gli altri. Questo perchè, parlando col committente, risulta che il robot sarà probabilmente implementato su componenti dedicati.

```
QActor cargorobot context ctx_robot {  
    State s0 initial {  
    }  
    Goto work  
    State work {  
        println("cargorobot | WORKING: moving containers from IOPort to slot5, then to reserved slot")  
    }  
}
```

Cargoservice

Il **cargoservice** è il macro-componente che implementa il core-business e orchestra tutti gli altri componenti, modellato quindi come **ATTORE**.

La logica di business principale è composta da:

- dove e quando spostare il cargorobot
- accettare o rifiutare una richiesta di carico
- quale slot libero associare alla richiesta che si sta servendo
- mandare out of order il sistema quando si accorge di un malfunzionamento del sensor

```
QActor cargoservice context ctx_cargoservice {  
    //dai requisiti, gli stati del sistema sono: ENGAGED, DISENGAGED, OUTOFSERVICE  
    State s0 initial {  
        //inizializzare hold  
    }  
}
```

```

Goto disengaged

State disengaged {
    println("cargoservice | DISENGAGED: aspetto load_request") color green
}
Transition t0
    whenRequest load_request -> engaged

State engaged {
    //controllo: hold piena? -> load_rejected
    //                outOfService? -> retrylater
    //                engaged? -> retrylater
    //                tutto okay? -> load_accepted[slot]

    //muovo robot a ioport
    //muovo robot a slot 5
    //muovo robot a slot 1..4 riservato
}
Goto disengaged

State outOfService {
    //gestisco out of service
}
}

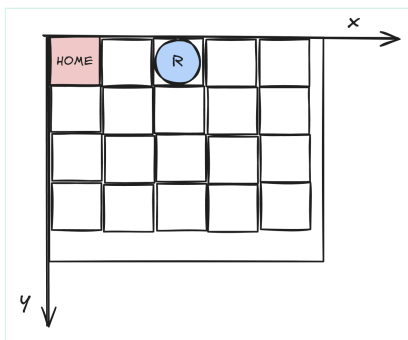
```

[Link al modello QAK](#)

Hold e slots

La **hold** è lo spazio di lavoro del cargorobot. Si tratta di uno spazio rettangolare di altezza **H** e base **B** (deducibili da figura nei requisiti), contenente dei punti di interesse per la logica di business, in particolare gli **slot**, anch'essi con una posizione definita (mostrata in figura da requisiti). La hold dai requisiti ci sembra un componente passivo, ampiamente legata alla logica dei dati (dimensione, slot e carichi). Decidiamo quindi di rappresentare la logica dei dati attraverso il POJO **Hold**.

Per rappresentare lo spazio rettangolare dell'hold e l'esatto posizionamento degli **slot** al suo interno, abbiamo pensato di utilizzare un sistema di coordinate discrete, ottenute suddividendo l'area della **hold** in quadrati. Questo si riflette nel modello come una matrice di interi **int[][] holdMap**.



Da requisiti, ogni **slot** può essere riservato o libero, perciò abbiamo pensato di modellare **slot** come una classe **Slot**, dotata di un parametro boolean **reserved** corrispondente agli stati (true = riservato, false = libero), e dei metodi **free**, **reserve**, **isReserved**.

Decidiamo di modellare la posizione dei punti di interesse **slot** come indici interi $i[1 \dots n+1]$, che verranno inseriti all'interno della matrice, mentre gli spazi "vuoti" di **holdMap** verranno modellati come 0.

```

public Class Slot{
    private boolean reserved;

    public void free(){
        reserved = false;
    }

    public void reserve(){
        reserved = true;
    }
}

```

```

    public boolean isReserved(){
        return reserved;
    }
}

public Class Hold{
    private int[][] holdMap;
    private int width;
    private int height;
    private int number_of_slots n;
    private Slot slots[];

    public void setWidthHeight(int B, int H){
        //inizializzazione dimensioni dell'hold da dati in input
        width = B;
        height = H;
        holdMap = new int[H][W];

        //inizializzazione dello spazio dell'hold a 0, quindi vuoto
        for(int i = 0; i < height; i++){
            for(int j = 0; j < width; j++){
                holdMap[i][j] = 0;
            }
        }
    }

    public void setSlots(int n, int[][] slots_coordinates){
        this.n = n;
        slots = new Slot[n];

        //posizionamento statico degli slots e inizializzazione reservation a false, free
        for(int i = 0; i < n; i++){
            int slot_x = slots_coordinates[n][0];
            int slot_y = slots_coordinates[n][1];

            holdMap[slot_y][slot_x] = n+1;

            slots[i].free();
        }
    }

    public void freeSlot(int slot_id){
        slots[slot_id - 1].free();
    }

    public void reserveSlot(int slot_id){
        slots[slot_id - 1].reserve();
    }

    public boolean isSlotReserved(int slot_id){
        return slots[slot_id - 1].isReserved();
    }
}

```

IOPort

La **IOPort** è fisicamente il punto di ingresso/uscita dei container nella hold.

Natura software: Come descritto dal committente, **pushbutton** e **display** verranno implementati in una **interfaccia web**, che decidiamo di rappresentare attraverso un attore QAK chiamato **ioport** con un contesto dedicato.

La IOPort è responsabile della comunicazione con l'utente tramite:

- **pushbutton**: pulsante virtuale che, quando premuto dal cliente, invia una **richiesta di carico** al cargoservice.
- **display**: è un componente che, nelle modalità descritte nei requisiti, deve essere in grado di mostrare sull'interfaccia web:
 - La risposta alla richiesta di carico conseguente alla pressione di **pushbutton**: "Accepted" oppure "Retry Later"
 - Lo "stato corrente" della **hold**
 - Lo stato di funzionamento del **sensor**: "Service working" oppure "Out of Service"

Dall'analisi dei requisiti iniziamo a modellare alcuni messaggi scambiati tra **cargoservice** e **ioport**, con lo scopo di inviare la richiesta di carico da parte dell'utente alla pressione di **pushbutton**:

```

Request load_request : load_request(X)
Reply  load_accepted : load_accepted(SLOT) for load_request
Reply  load_rejected : load_rejected(X)   for load_request
Reply  retrylater    : retrylater(X)      for load_request

```

```

QActor ioport context ctx_ioport {
  State s0 initial {
    //in attesa della pressione del pushbutton
  }
  Goto pushbutton

  State pushbutton {
    println("pushbutton pressed")
    request cargospace -m load_request : load_request(X)
  }
  Transition to
    whenReply load_accepted -> accepted
    whenReply load_rejected -> rejected
    whenReply retrylater -> retrylater

  State accepted {
    println("ioport | DISPLAY: load accepted") color green
  }
  Goto s0

  State retrylater {
    println("ioport | DISPLAY: retry later") color yellow
  }
  Goto s0

  State rejected {
    println("ioport | DISPLAY: load rejected") color red
  }
  Goto s0
}

```

[Link al modello QAK](#)

Sensor

Il **sensor** è un componente **attivo** che rileva la presenza o l'assenza di un container presso la IOPort, misurando una distanza D.

Inoltre, data una distanza DFREE, valgono le seguenti proprietà:

$D \leq DFREE/2$	= container presente, IOPort occupata
$DFREE/2 \leq D \leq DFREE$	= misurazione valida, IOPort libera
$D \geq DFREE$ per più di 3 sec	= sistema entra in Out-of-Service

Natura software: **attore QAK** con un proprio contesto **ctx_sensor**, poichè eseguito su un Raspberry che deve comunicare con il contesto **ctx_cargospace** del PC della Hold. (Ricordiamo il ruolo del [contesto in QAK](#)).

Led

Il **led** è un dispositivo di notifica visiva, presente sullo stesso Raspberry del sensor, come richiesto dal committente. I suoi comportamenti:

- BLINK: stato **ENGAGED**
- OFF: stati **DISENGAGED/OUT OF ORDER**

Modelliamo il led come **attore QAK**, e, a seconda dello stato di cargospace, il led verrà portato nello stato corrispondente nella sua macchina a stati.

```

QActor led context ctx_sensor {
  State s0 initial {}
  Goto off

  State off {
    println("led | DISENGAGED: led off")
  }

  State on {

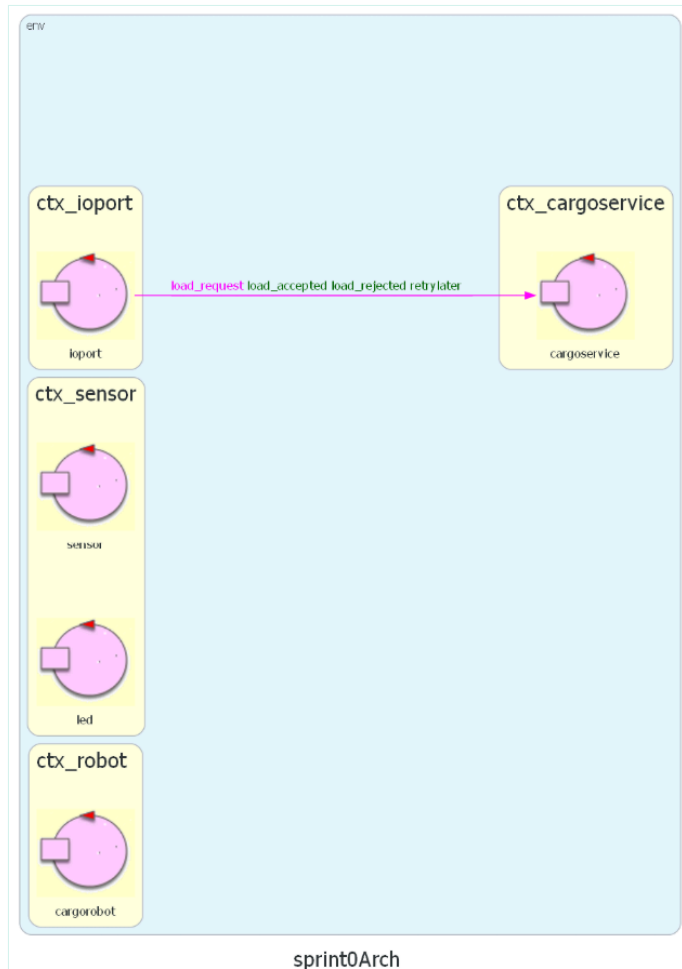
```

```
println("led | ENGAGED o OUT OF ORDER: led on")
}
}
```

Architettura Logica del Sistema

Definiamo una prima architettura logica del sistema, composta dai quattro attori principali **cargoservice**, **sensor**, **webGUI**, **robotsmart**, ognuno con il proprio contesto.

[Link al modello QAK](#)



Componenti da Sviluppare

Da sviluppare

- **hold** : Sprint 1 (gestione mappa e slot)
- **cargoservice**: Sprint 1 (core business, gestione stati)
- **sensor** : Sprint 2 (attività del sensore virtuale/hardware)
- **led** : Sprint 2 (notifica visiva virtuale/hardware)
- **ioport** : Sprint 3 (interfaccia Web)
- **pushbutton** : Sprint 3 (input virtuale in WebGUI)
- **display** : Sprint 3 (output verso webGUI)

Forniti dal committente

Essendo la nostra azienda specializzata in robotica, ci vengono forniti diversi software di controllo del DDR, ognuno con proprietà diverse.

Abbiamo due implementazioni di robot: [VirtualRobot26](#) e [RobotSmart26](#), queste si differenziano perchè virtual robot implementa solo movimenti "base", come muovere il robot in avanti o farlo ruotare, mentre robot smart

implementa anche il concetto di stato locale e di planning.

Inoltre abbiamo anche altri due componenti, ovvero [RobotObj26](#) e [RobotService26](#). La prima espone i comandi del robot tramite Interfaccia Java, quindi permettendo di vedere il robot come un POJO, mentre l'altra rende il robot un attore QAK, quindi un servizio. Entrambe servono a esporre comandi di alto livello riducendo l'abstraction gap e incapsulando il funzionamento del robot rendendolo trasparente al programmatore

Sarà compito dei progettisti decidere se e quale di questi software potrà essere usato per il progetto.

L'altro componente fornito è **WEnv**, un ambiente virtuale che simula la Hold e il robot, e ci permette di sviluppare l'interesse del sistema senza dipendere da un'implementazione fisica. ([Documentazione WEnv](#)). Questo ambiente virtuale è servito con Docker.

Test Plans

I test sono scritti in Java (JUnit). Sono dei test preliminari, verranno implementati nello Sprint1 e potrebbero cambiare sintatticamente.

Prima di ogni test il sistema è avviato e **cargoservice** è raggiungibile.

sensor è simulato: i diversi messaggi vengono iniettati direttamente dal tester, mentre uno **stub** di sensor risponde alle richieste di presenza del carico.

I messaggi scambiati sono definiti e descritti nella prima architettura QAK del nostro sistema (link al file su github).

T01: Richiesta accettata - ENGAGED

Precondizione: sistema in DISENGAGED, almeno uno slot1-4 libero.

Atteso: cargoservice risponde **load_accepted(slotN)** e passa a ENGAGED.

```
@Test
public void testT01() throws Exception {
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T01: load_accepted con slot assegnato", resp.contains("load_accepted"));
}
```

T02: Container depositato - normale operazione - DISENGAGED (normale flow di operazione)

Precondizione: sistema in ENGAGED (T01 eseguito), IOPort libera al momento della richiesta.

Atteso: dopo avere depositato il container alla IOPort, cargorobot esegue il ciclo IOPORT - SLOT5 - slotN, il sistema torna in DISENGAGED con lo slot occupato.

```
@Test
public void testT02() throws Exception {

    //porta il sistema in ENGAGED
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    conn.request(req);

    //simula sensor: container rilevato all'IOPort (dispatch asincrono)
    conn.forward(CommUtils.buildDispatch("tester", "containerIn", "containerIn(x)", "cargoservice").toString());

    CommUtils.delay(8000); //attendo finisca la normale operazione

    //una nuova richiesta deve essere accettata (sistema tornato in DISENGAGED)
    String req2 = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp2 = conn.request(req2);

    assertTrue("T02: sistema tornato in DISENGAGED dopo carico", resp2.contains("load_accepted"));
}
```

T03: Timeout in ENGAGED - DISENGAGED - slot liberato

Precondizione: sistema in ENGAGED, un solo slot libero.

Atteso: allo scadere dei 30s senza avere depositato un container alla IOPort, il sistema torna in DISENGAGED con lo slot liberato (prima riservato) e risponde **retrylater** alla richiesta originale.

```
@Test
public void testT03() throws Exception {
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);
    assertTrue("T03 setup: richiesta accettata", resp.contains("load_accepted"));

    //non piazza un container, attendo scadenza timer (5 secondi per testare)
    CommUtils.delay(6000);

    //lo slot deve essere stato liberato
    String req2 = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp2 = conn.request(req2);

    assertTrue("T03: slot liberato dopo timeout", resp2.contains("load_accepted"));
}
```

T04: Hold piena - load_rejected

Precondizione: tutti gli slot1-4 occupati.

Atteso: cargoservice risponde **load_rejected**.

```
@Test
public void testT04() throws Exception {

    //invio richiesta di carico
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T04: hold piena - load_rejected", resp.contains("load_rejected"));
}
```

T05: Sensor guasto - Out of Service - retrylater

Precondizione: sistema in DISENGAGED.

Atteso: dopo **sensorError** da sensor il sistema entra in OUT_OF_SERVICE, le richieste successive ricevono **retrylater(out_of_service)**.

```
@Test
public void testT05() throws Exception {
    //simulo sensor guasto
    conn.forward(CommUtils.buildDispatch("tester", "sensorError", "sensorError(x)", "cargoservice").toString());
    CommUtils.delay(300);

    //invio richiesta di carico
    String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
    String resp = conn.request(req);

    assertTrue("T05: out_of_service - retrylater", resp.contains("retrylater"));
}
```

T06: Container già presente all'IOPort - carico immediato

Precondizione: container depositato **prima** della load_request

Atteso: alla load_request, cargoservice interroga sensor e il ciclo di movimentazione parte subito, senza attendere i 30s.

```
@Test
public void testT06_ContainerAlreadyAtIoport() throws Exception {
    //setto lo stub del sensoreservice in stato "container presente"
    sensorStub.setContainerStatus(true);
}
```

```

//invio richiesta di carico
String req = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
String resp = conn.request(req);
assertTrue("T06 setup: richiesta accettata", resp.contains("load_accepted"));

CommUtils.delay(8000); //attendo finisca l'operazione

//sistema tornato in DISENGAGED: nuova richiesta accettata
sensorStub.setContainerStatus(false);
String req2 = CommUtils.buildRequest("tester", "load_request", "load_request(x)", "cargoservice").toString();
String resp2 = conn.request(req2);

assertTrue("T06: carico avviato con container già presente", resp2.contains("load_accepted"));
}

```

Goal Sprint 1

Lo **Sprint 1** ha come obiettivo la realizzazione del core business, per ora senza dipendere da IOPort e dal sensore.

Sottoinsieme di Requisiti (goal Sprint 1)

- Implementare **cargoservice**
- Implementare Hold e Slot
- Implementare la comunicazione con **cargorobot**
- Gestire la riserva degli slot e il ciclo di movimentazione
- Implementare sensor simulato (stub di sensor)
- Gestire la verifica della presenza di container presso IOPort
- Gestire il timeout di 30 secondi per il deposito

Test da superare in Sprint 1

- **T01**: Richiesta accettata
- **T02**: Container depositato in tempo
- **T03**: Timeout / DISENGAGED
- **T04**: Hold piena
- **T05**: Sensor guasto / OUT_OF_SERVICE
- **T06**: Container già presente all'IOPort

Componenti prodotti in Sprint 1

- **cargoservice**
- **Hold** e **Slot**
- Mock **sensor**

Piano di lavoro

- Prevediamo circa 20 ore/uomo, su una settimana di lavoro.
- Francesco si occuperà principalmente del modello QAK.
- Andrea si occuperà principalmente del testing e stesura del report.
- Entrambi ci aggiorniamo giornalmente sul lavoro svolto, studiamo e valutiamo i rispettivi lavori.



By Andrea Gazzola

and Francesco Roffia



[GIT](#)

repo: https://github.com/FRoffia/ISS_26_Cargoservice